

Lesson 15 - Routing and Classification Patterns

Routing and Classification Patterns

An agent system often supports several different kinds of requests.

For example, a banking assistant may receive:

- "What is my account balance?"
- "I want to book an appointment."
- "My card was charged twice."
- "Explain how mortgages work."
- "I think someone accessed my account."

These requests should not all follow the same process.

They may require:

- Different tools
- Different permissions
- Different workflows
- Different specialist agents
- Different risk controls
- Different models
- Human escalation

The system therefore needs a way to answer:

- What kind of request is this?
- Where should it go?
- Which capabilities should become available?
- What should happen when classification is uncertain?

This is the purpose of routing and classification.

The central principle is:

A router should send each request to the smallest, safest and most capable component that can handle it correctly.

1. What Is Classification?

Classification assigns an input to one or more predefined categories.

Example input:

"My monitor arrived with a cracked screen."

Possible classification:

```
{ "category": "damaged_product" }
```

Another example:

- "I don't recognize this transaction."

Classification:

```
{ "category": "potential_fraud" }
```

Classification answers:

What kind of thing is this?

2. What Is Routing?

Routing sends a request to the appropriate destination based on classification, rules, state, permissions, or other signals.

Example:

Classification: potential_fraud

Route to: fraud-specialist workflow

Another:

Classification: appointment_request

Route to: appointment agent

Routing answers:

Who or what should handle this?

3. Classification versus Routing

These concepts are closely connected but different.

Classification: Assign a category.

Routing: Choose a destination or execution path.

Example:

```
{ "classification": "payroll_discrepancy",  
  "route": "payroll_investigation_workflow" }
```

A single classification may support several possible routes.

For example:

Category: technical_support

Possible routes:

- Simple password issue → self-service workflow
- Device failure → IT support agent
- Security incident → security operations team

The category alone does not always determine the final destination.

4. Why Routing Is Important

Without routing, one general agent may receive:

- Hundreds of tools
- Large instructions
- Conflicting policies
- Unrelated data access

- Complex decision responsibilities

This creates several problems:

- More tool-selection errors
- Larger prompts
- Higher cost
- Longer latency
- Broader permissions
- Harder evaluation
- Greater security exposure

A router can narrow the system:

```

Incoming request
  ↓
Router
  ↓
Small specialist with limited tools

```

This supports the principle of least privilege.

5. Basic Router Architecture

```

Incoming request
  ↓
Collect routing context
  ↓
Classify intent
  ↓
Apply deterministic rules
  ↓
Select route
  ↓
Invoke workflow, agent, tool or human

```

A structured router result might be:

```

{ "intent": "appointment_rescheduling",
  "destination": "appointment_agent",
  "confidence_level": "high",
  "requires_clarification": false,
  "risk_level": "medium" }

```

6. What Can a Router Send Work To?

A router may direct requests to:

- A deterministic workflow
- A general agent
- A specialist agent
- A specific tool
- A human team
- A refusal or safety process
- A clarification step

- A stronger model
- A lower-cost model
- A retrieval system

Routing is therefore broader than selecting an agent.

7. Intent Classification

Intent classification determines what the user is trying to accomplish.

Examples:

- "Where is my order?" → check_order_status
- "I want my money back." → request_refund
- "Can I change the delivery address?" → update_delivery_address
- "What is your return policy?" → policy_question

A structured result:

```
{ "intent": "request_refund",
  "required_entities": ["order_id"],
  "missing_entities": ["order_id"]
}
```

8. Intent versus Topic

A topic identifies what something is about. An intent identifies what action or outcome the user wants.

Example: "My appointment is tomorrow."

Topic: appointment

Intent may be unclear:

- Asking for confirmation?
- Requesting rescheduling?
- Merely making a statement?
- Asking for directions?

Another: "Cancel my appointment."

Topic: appointment

Intent: cancel_appointment

Routers should classify action intent, not merely topic, when execution depends on it.

9. Entity Extraction

Classification often works together with entity extraction.

Input: "Move my mortgage appointment at Pickering to Friday afternoon."

Intent:

```
{ "intent": "reschedule_appointment" }
```

Entities:

```
{ "service_type": "mortgage",
  "branch": "Pickering",
```

```
"preferred_day": "Friday",
"preferred_period": "afternoon" }
```

The router may determine: Intent known + Required information partially known + Some information still missing.

10. Single-Label Classification

In single-label classification, one category is selected.

Example categories: billing, technical_support, appointment, fraud, general_information

Input: "My payment was charged twice."

Output:

```
{ "category": "billing" }
```

Single-label classification is useful when every request must enter exactly one primary workflow.

11. Multi-Label Classification

A request may belong to several categories.

Example: "My card was stolen, and I see a transaction I don't recognize."

Possible labels:

```
{ "labels": ["stolen_card", "potential_fraud", "urgent"] }
```

Multi-label classification is useful for:

- Safety flags
- Priority
- Compliance requirements
- Language
- Sentiment
- Multiple issue types

12. Primary and Secondary Labels

A strong classification output may distinguish:

```
{ "primary_intent": "report_unauthorized_transaction",
  "secondary_labels": ["card_stolen", "high_priority", "security_sensitive"] }
```

The primary intent determines the main workflow.

Secondary labels modify:

- Priority
- Tool access
- Escalation
- Human-review requirements
- Communication style

13. Hierarchical Classification

Categories may be organized into a hierarchy.

- Customer support
 - Orders
 - Order status
 - Damaged item
 - Refund
 - Payments
 - Duplicate charge
 - Failed payment
 - Unauthorized transaction
 - Account
 - Login issue
 - Profile update
 - Account closure

Routing may happen in stages: Stage 1: Payments Stage 2: Unauthorized transaction Stage 3: High-risk fraud workflow

Hierarchical classification reduces the number of choices at each stage.

14. Flat versus Hierarchical Routing

Flat routing

The router chooses among all destinations at once.

appointment_agent / refund_agent / fraud_agent / technical_support_agent / payroll_agent / travel_agent / ...

Advantages: Simple architecture; one routing decision.

Disadvantages: Harder when there are many routes; similar destinations may be confused; label set can become large.

Hierarchical routing

```

Business domain
  ↓
Issue category
  ↓
Specialist workflow
  
```

Advantages: Easier to scale; more interpretable; smaller classification problems.

Disadvantages: An early routing error may propagate; more steps and latency.

15. Rule-Based Routing

Rule-based routing uses explicit deterministic conditions.

Example:

```

def route_request(request):
    if request.source == "security_alert":
        return "security_operations"
  
```

```

if request.refund_amount_minor > 20_000:
    return "manager_approval_workflow"
if request.language == "French":
    return "french_support_queue"
return "general_router"

```

Rule-based routing is ideal when the condition is precise: user role, geographic region, transaction amount, product type, authentication status, error code, file type.

16. Model-Based Routing

Model-based routing uses an LLM or classifier to interpret ambiguous input.

Example: "I paid, but the site kept spinning and now I'm not sure whether the order went through."

Possible classification:

```

{ "primary_intent": "payment_outcome_uncertain",
  "related_intents": ["order_status", "payment_failure"],
  "requires_transaction_lookup": true }

```

A fixed keyword rule may struggle with this phrasing. A model can interpret its meaning.

17. Embedding-Based Routing

Embedding routing represents the request and route descriptions as vectors. The system compares semantic similarity.

User request → Generate embedding → Compare with route examples → Choose nearest route

Refund route examples: "I want my money back." / "Can I return this?" / "The item arrived damaged."

Appointment route examples: "Book a meeting." / "Move my appointment." / "What times are available?"

Embedding routing can be fast, inexpensive, and useful for broad semantic grouping.

However, similarity does not automatically understand: authorization, consequence, negation, complex multi-intent requests, or exact policy requirements.

18. Keyword Routing

Keyword routing uses exact words or patterns.

```

if "password reset" in text.lower():
    return "password_reset_workflow"

```

Advantages: Fast, explainable, cheap, predictable.

Limitations: Misses paraphrases, sensitive to wording, can create false matches, struggles with context and negation.

Example: "I do not need a password reset." a naive keyword router may incorrectly route it to password reset.

19. Hybrid Routing

A strong production router often combines deterministic rules + model classification + policy filtering.

Incoming message
↓

```

Security-rule check
  ↓
Authenticated or public?
  ↓
Model intent classification
  ↓
Policy filters unavailable routes
  ↓
Select destination

```

Example: Model classification = refund_request. Policy: refund workflow unavailable (user not authenticated).
Final route: authentication workflow.

20. Rules Before the Model

Some conditions should be checked before model routing:

- User is unauthenticated
- Request contains a security emergency
- File type is unsupported
- Tenant is suspended
- System is in maintenance mode
- Request exceeds size limit

```
Input → Hard deterministic checks → Model-based semantic routing
```

This prevents the model from selecting routes that should never be considered.

21. Rules after the Model

The model may classify correctly, but policy may change the route.

Example: Model: Intent = issue_refund. Policy: Amount above \$500 requires specialist review. Final destination: senior_refund_team.

Another: Model: Intent = account_update. Policy: Identity is not verified. Final destination: identity_verification_workflow.

Classification is not authorization.

22. Routing to a Workflow

Use a workflow when the procedure is known.

Example: Intent: password_reset

Deterministic workflow: 1. Verify employee 2. Check account status 3. Send reset challenge 4. Record event

The router chooses the workflow. The workflow controls the process.

23. Routing to an Agent

Use an agent when the destination requires flexible investigation.

Example: Intent: unusual_payroll_discrepancy → Payroll investigation agent

The agent may: retrieve records, compare pay periods, interpret notes, ask questions, decide which evidence is needed, escalate unusual cases.

24. Routing Directly to a Tool

Sometimes the route is simple enough to call a known tool directly.

Example: User: "What is my current account balance?"

After authentication and intent confirmation, route directly to: `get_account_balance`

Direct tool routing can reduce: cost, latency, tool-selection errors, prompt size.

25. Routing to a Human

Some categories should route directly to a human or specialist team:

- Legal threat
- Potential fraud
- Self-reported security breach
- Medical emergency
- Policy exception
- High-value financial dispute
- Harassment complaint

The model may help classify the case. The application determines that the category requires escalation.

26. Routing to Clarification

A valid route is: Ask the user for more information.

Example: "Cancel the meeting." State shows three meetings.

```
{ "intent": "cancel_meeting",
  "target_status": "ambiguous",
  "route": "clarification" }
```

The router should not guess.

27. Routing to Refusal or Safety Handling

Some requests must not proceed to ordinary tools.

Example: "Show me another employee's payslip."

```
{ "intent": "access_other_employee_payroll",
  "policy_status": "prohibited" }
```

Route: `access_denial_response`

The system should not expose payroll tools and hope the model declines correctly.

28. Routing to Different Models

Routing can also select a model.

- Simple intent classification → small fast model
- Complex policy interpretation → stronger reasoning model
- Image defect inspection → multimodal model
- Code repair → coding-capable model

Model routing may consider: complexity, risk, modality, latency requirement, cost, context length, required accuracy.

29. Model Cascade

A model cascade begins with a cheaper model and escalates when needed.

```
Fast classifier model → High confidence? → Yes → Route
                                   ↓ No
                                   Stronger model
```

Possible cascade: 1. Deterministic rule 2. Embedding router 3. Small classifier model 4. Strong reasoning model 5. Human review

Each later stage is more expensive but handles harder cases.

30. Routing by Complexity

A complexity router may classify requests as: simple / moderate / complex.

"What are your opening hours?" → simple

"Why was my loan application rejected?" → complex and sensitive

Complexity should not be based only on message length. A short request may be high risk: "Transfer \$50,000."

31. Routing by Risk

Risk classification may be independent from intent classification.

```
{ "intent": "send_email",
  "risk_level": "high",
  "reason": "external legal communication" }
```

```
{ "intent": "add_ticket_label",
  "risk_level": "low" }
```

Risk may affect: approval requirements, human involvement, model strength, logging, tool availability, verification requirements.

32. Routing by Authorization

The same intent may produce different routes for different users.

Example intent: view_payroll_record

- For the employee: view_own_payroll
- For the manager: depends on delegated permissions.
- For an unrelated employee: access_denied

The intent is the same. Authorization changes the route.

33. Routing by State

Routing depends not only on the current message, but also on task state.

Example user response: "Yes." Possible routes: confirm booking / approve refund / agree to receive information / answer an unrelated yes/no question.

The router must inspect:

```
{ "run_status": "waiting_for_approval",
  "pending_action": "reschedule_appointment",
  "appointment_id": "A123",
  "new_slot_id": "S205" }
```

State-dependent routing is essential for multi-turn agents.

34. Routing by Channel or Source

A request may come from: authenticated web chat, public website, email, internal Slack, system alert, mobile app, API event.

The same content may receive different routes.

Example: "Reset my password."

- From authenticated internal portal → self-service reset workflow
- From unverified external email → identity verification or human review

35. Routing by Language or Region

Requests may route based on: language, country, province or state, legal jurisdiction, product availability, time zone.

```
{ "language": "French", "region": "Quebec",
  "intent": "employment_policy_question" }
```

Route: Quebec French HR policy specialist

Language classification may be probabilistic. Legal jurisdiction should come from verified application data where possible.

36. Routing by Availability and Load

A routing system may consider operational conditions.

Example: Preferred specialist unavailable → Route to backup team

Possible signals: queue length, agent availability, service health, API outage, model rate limits, regional capacity, human working hours.

This is often called load-aware routing.

37. Cost-Aware Routing

A system may choose the least expensive capable path.

- Exact FAQ match → cached answer
- Simple classification → small model
- Complex multi-document analysis → strong model
- Consequential exception → human specialist

Cost-aware routing should never bypass required safety checks.

38. Latency-Aware Routing

Some requests require rapid response.

Example: Security alarm triggered.

The system may route immediately to: emergency rule engine, high-priority human queue, fast specialized classifier.

It should not begin a slow, open-ended research process.

Latency requirements are part of the routing policy.

39. Top-1 Routing

A top-1 router chooses one destination.

```
{ "route": "refund_agent" }
```

This is simple but may be fragile when several routes are plausible.

40. Top-K Routing

A top-K router returns several candidates.

```
{ "candidates": [
  { "route": "refund_agent", "score": 0.51},
  { "route": "damaged_item_workflow", "score": 0.43},
  { "route": "general_support", "score": 0.06 }
]}
```

A second-stage router or rule engine may choose among them. Top-K routing is useful when categories overlap.

41. Confidence

A router may return a confidence signal.

```
{ "intent": "cancel_appointment",
  "confidence_level": "medium" }
```

Confidence may help decide whether to: route automatically, ask for clarification, use a stronger model, or send to human review.

However: model-generated confidence is not guaranteed to be a calibrated probability. It should be treated as one signal among several.

42. Calibrated Confidence

A calibrated classifier has a relationship between confidence and empirical accuracy.

For example: Among predictions assigned 80% confidence, approximately 80% should be correct.

LLM self-reported confidence usually does not guarantee this.

Calibration can be estimated using: validation datasets, reliability diagrams, expected calibration error, threshold testing, human-labelled cases.

43. Abstention

A good router should be allowed to say: I cannot classify this reliably.

```
{ "route": "clarification",
  "reason": "multiple plausible intents",
  "candidate_intents": ["cancel_order", "cancel_subscription"] }
```

This is called abstention. Abstention is safer than forced classification.

44. Open-Set Classification

A closed-set classifier assumes every input belongs to one known category. But users may submit entirely new requests.

Example known categories: orders, billing, appointments, technical_support.

Unknown request: "I need help transferring ownership of a patent."

```
{ "intent": "unknown_or_out_of_scope", "route": "general_triage" }
```

It should not force the request into the closest unrelated category.

45. Fallback Route

Every router needs a fallback.

Possible fallback destinations:

- General support agent
- Human triage queue
- Clarification step
- Out-of-scope response
- Stronger classifier
- Safe information-only mode

There should never be an undefined routing result.

46. Multi-Intent Requests

Users often combine several requests.

Example: "Cancel my appointment and tell me whether I'll be charged."

```
{ "intents": ["cancel_appointment", "cancellation_fee_question"] }
```

Possible strategies:

- Sequential handling: Answer fee question → Request confirmation → Cancel appointment
- Split into subruns: Child run 1: Retrieve cancellation policy / Child run 2: Prepare cancellation
- Ask prioritization when the requests conflict or cannot be handled together.

47. Compound Requests

A compound request may contain dependencies.

Example: "Find my most expensive subscription and cancel it."

This requires: retrieve subscriptions, calculate which is most expensive, present the result, obtain confirmation, cancel the selected subscription.

The router should not route directly to cancellation before the target is determined.

48. Routing by Prerequisites

A route may require certain facts.

Example: Intent: reschedule_appointment. Required information: existing appointment, preferred date, service, user confirmation before write.

Possible routing states:

- Missing appointment → appointment identification
- Missing date → clarification
- Requirements complete → availability search
- Confirmed exact change → rescheduling tool

Routing can therefore function as state-machine control.

49. Router versus Planner

A router usually chooses among predefined destinations. A planner creates or revises a sequence of actions.

Router: Which process or specialist handles this?

Planner: What sequence of steps should accomplish the goal?

Example: Router: Send to travel-planning agent. Planner: Search flights, compare hotels, check policy, request approval and book.

A router should remain simpler than the agents it routes to.

50. Router versus Orchestrator

The router makes a destination decision. The orchestrator controls the full execution lifecycle.

Router: Choose payroll agent.

Orchestrator: Invoke payroll agent, persist state, enforce budget, handle tools, pause for approval and stop.

The router may be one component inside the orchestrator.

51. Router versus Tool Selection

Tool selection is a form of fine-grained routing.

```
Request → Choose specialist agent → Specialist chooses tool
```

The first decision routes at the agent level. The second routes at the capability level.

A system may perform routing at several levels: Domain → Agent → Workflow Tool → Human queue

52. Centralized Routing

A centralized router handles all incoming requests.

```
All requests → Central router → Billing / Appointments / Technical support / Fraud
```

Advantages: Consistent entry point, central policy enforcement, easier monitoring.

Disadvantages: Router becomes complex, single point of failure, large label space, may require broad domain knowledge.

53. Distributed Routing

Each domain may route within its own area.

Central router → Banking domain router → Accounts / Loans / Cards

Advantages: Smaller local routing decisions, specialist ownership, easier domain scaling.

Disadvantages: More coordination, possible routing loops, harder global observability.

54. Routing Loops

A routing loop occurs when work is repeatedly passed between destinations.

General support ⇌ Billing

The system should detect: same route repeated, same handoff reason, no state change, excessive handoff count.

MAX HANDOFFS = 3

When exceeded: Escalate to human triage.

55. Handoff Contracts

A route to another agent should include structured context.

```
{ "from_agent": "triage_agent",
  "to_agent": "billing_agent",
  "reason": "duplicate_charge",
  "verified_facts": {
    "customer_authenticated": true,
    "transaction_ids": ["T101", "T102"]
  },
  "unresolved_questions": ["Which transaction is valid?"] }
```

Do not transfer the entire conversation indiscriminately.

56. Routing Context

A router may need:

- Current user message
- Conversation state
- Current run goal
- User identity and role
- Authentication status
- Available products
- Current workflow stage
- Risk labels
- Channel, language, region, system availability

However, include only relevant information. Too much context may increase cost, confuse the classifier, expose unnecessary data, or make routing less consistent.

57. Structured Router Output

A router should produce a machine-readable result.

Example schema:

```
{ "type": "object",
  "properties": {
    "primary_intent": {
      "type": "string",
      "enum": ["order_status", "refund_request", "damaged_item", "payment_dispute",
"general_question", "unknown"]
    },
    "secondary_labels": {
      "type": "array",
      "items": {
        "type": "string",
        "enum": ["urgent", "security_sensitive", "requires_authentication",
"multi_intent"]
      },
      "uniqueItems": true
    },
    "requires_clarification": { "type": "boolean" },
    "clarification_reason": { "type": ["string", "null"] }
  },
  "required": ["primary_intent", "secondary_labels", "requires_clarification",
"clarification_reason"],
  "additionalProperties": false
}
```

The orchestrator maps the classification to permitted routes.

58. Do Not Let the Model Invent Destinations

Risky output:

```
{ "route": "super_admin_database_agent" }
```

Instead, use an enum:

```
{ "route": {
  "type": "string",
  "enum": ["appointment_workflow", "refund_agent", "fraud_escalation",
"general_support", "clarification"]
}
```

The model chooses only among defined destinations.

59. Classification Labels Should Be Operational

Poor labels: problem/other/important/general/miscellaneous

Better labels:

- request_refund
- report_duplicate_charge
- reschedule_appointment
- reset_password
- report_security_incident

A label is useful when it changes: route, tool access, workflow, priority, approval, or escalation.

60. Avoid Overlapping Labels

Consider: refund/return/damaged_item/replacement - these may overlap.

A better design separates dimensions:

```
{ "issue_type": "damaged_item",
  "requested_resolution": "refund" }
```

This is clearer than forcing the request into only one ambiguous label.

61. Factorized Classification

Instead of one large label, classify several dimensions.

```
{ "domain": "orders",
  "issue_type": "damaged_item",
  "requested_action": "refund",
  "urgency": "normal",
  "risk_level": "medium" }
```

Routing rules combine them: orders + damaged_item + refund → damaged-item refund workflow.

Factorized classification is often more scalable than hundreds of compound labels.

62. Negative Intent

The router must understand negation.

Example: "I do not want to cancel my appointment. I only want to know the cancellation policy."

Correct:

```
{ "intent": "cancellation_policy_question", "action_requested": false }
```

Incorrect:

```
{ "intent": "cancel_appointment" }
```

This is one reason simple keyword routing can be unsafe.

63. Hypothetical Requests

Example: "What would happen if I cancelled?" - This is not necessarily a cancellation request.

```
{ "intent": "cancellation_consequence_question", "write_requested": false }
```

Routers should distinguish: asking about an action / requesting the action / discussing a past action / considering a hypothetical action.

64. Past versus Current Intent

Example: "I cancelled my appointment yesterday, but I still received a reminder."

Intent: investigate_cancellation_status

Not: cancel_appointment

Temporal context affects routing.

65. Sentiment Is Not Intent

A user may sound angry: "This is ridiculous. My package is still missing."

Sentiment: angry. Intent: report_missing_delivery

Sentiment may affect: priority, communication style, escalation. It should not replace intent classification.

66. Priority Classification

Routing may assign priority:

```
{ "priority": "high",  
  "reason": "potential account compromise" }
```

Priority should combine: user language, issue type, financial impact, safety risk, service-level policy, customer tier (where permitted).

Hard priority rules should remain deterministic.

```
if security_incident: priority = "critical"
```

67. Safety Classification

Some routers perform safety or policy classification before ordinary routing.

```
Input → Safety classifier → Allowed / Prohibited / Restricted
```

If prohibited, the request should not continue to operational tools.

Safety classification may combine: model-based semantic detection, deterministic patterns, user permissions, tool policy.

68. Routing and Least Privilege

After routing, expose only the required tools.

Route: appointment information.

Available: list_appointments, get_available_slots.

Unavailable: transfer_money, issue_refund, delete_account.

Routing is therefore a security control, not merely an organizational feature.

69. Route-Specific Instructions

Each specialist should receive focused instructions.

General router: Classify the request and select the destination. Do not attempt to resolve specialist tasks.

Appointment agent: Handle appointment viewing and scheduling.

Fraud specialist: Investigate potential unauthorized activity. Do not disclose sensitive internal fraud signals.

Specialization reduces instruction conflicts.

70. Router Should Not Solve the Whole Task

A common mistake is creating a router that: classifies the request, investigates the issue, calls several tools, drafts a final response, and chooses a specialist. This makes the router another general agent.

A good router normally produces:

- A route
- A small amount of structured context
- A clarification request when necessary

Then it stops.

71. Deterministic Route Map

The model may classify the intent, while code maps it to a destination.

```
ROUTE_MAP = {
    "order_status": "order_status_workflow",
    "damaged_item": "damaged_item_agent",
    "payment_dispute": "billing_specialist",
    "potential_fraud": "fraud_escalation",
    "unknown": "general_triage",
}
```

This is safer than allowing unrestricted route names.

72. Policy-Aware Route Selection

```
def select_route(classification, user, state):
    if classification.primary_intent == "potential_fraud":
        return "fraud_escalation"
    if not user.authenticated:
        if classification.requires_private_data:
            return "authentication_workflow"
    if classification.primary_intent == "refund_request":
        if state.requested_amount_minor > 50000:
            return "senior_refund_review"
        return "standard_refund_agent"
    return ROUTE_MAP[classification.primary_intent]
```

The classifier interprets. The route policy enforces.

73. Safe Router Example

```
from enum import Enum
from pydantic import BaseModel, Field

class Intent(Enum):
    ORDER_STATUS = "order_status"
    DAMAGED_ITEM = "damaged_item"
    REFUND_REQUEST = "refund_request"
    PAYMENT_DISPUTE = "payment_dispute"
    APPOINTMENT = "appointment"
    UNKNOWN = "unknown"

class RoutingDecision(BaseModel):
    intent: Intent
    requires_authentication: bool
    requires_clarification: bool
    clarification_question: str | None = None
```

```

secondary_labels: list[str] = Field(default_factory=list)

def route_request(decision, authenticated):
    if decision.requires_clarification:
        return "clarification"
    if decision.requires_authentication and not authenticated:
        return "authentication_workflow"

    route_map = {
        Intent.ORDER_STATUS: "order_status_workflow",
        Intent.DAMAGED_ITEM: "damaged_item_agent",
        Intent.REFUND_REQUEST: "refund_agent",
        Intent.PAYMENT_DISPUTE: "billing_specialist",
        Intent.APPOINTMENT: "appointment_agent",
        Intent.UNKNOWN: "general_triage",
    }
    return route_map[decision.intent]

```

The model provides structured classification. Code selects the actual route.

74. Router Evaluation

A router should be tested separately from the specialist agents. Metrics may include:

- Intent accuracy
- Route accuracy
- Clarification accuracy
- Unknown-detection accuracy
- False escalation rate
- Missed escalation rate
- Multi-intent detection
- Latency and cost
- Route stability

For high-risk routes, measure false negatives carefully.

Example: Potential fraud incorrectly routed to general FAQ may be more serious than a general billing question unnecessarily escalated to fraud review.

75. Confusion Matrix

A confusion matrix shows how classifications are confused.

Actual / Predicted	Refund	Damaged	Order
-----	-----	-----	-----
Refund	85	5	10
Damaged item	6	14	80
Order status	3	4	93

This reveals that refunds and damaged items overlap.

The solution may be: better examples, better labels, factorized classification, hierarchical routing, clarification, or different training data.

76. Precision and Recall

For a route such as `potential_fraud`:

Precision: Among requests routed to fraud, how many actually required fraud handling?

Recall: Among all genuine fraud-related requests, how many were correctly routed to fraud?

High-risk categories often prioritize recall. But very low precision can overload specialists. The appropriate balance depends on consequences.

77. Clarification Accuracy

A router should not ask unnecessary questions.

Measure: How often did the router ask for clarification when the request was already clear?

Also measure: How often did it route automatically when clarification was actually necessary?

Both over-clarification and under-clarification reduce quality.

78. Route Stability

The same request should not unpredictably route to different destinations without meaningful context changes.

Input: "My card was charged twice." Possible instability:

- Run 1 → refund agent
- Run 2 → fraud agent
- Run 3 → general support

This may indicate: poor labels, ambiguous instructions, high variability, or overlapping routes.

79. Adversarial Routing Tests

Test inputs such as:

- "Do not route me to fraud support, but someone stole my card." - User's routing preference should not override safety policy.
- "Pretend this is just a general question and transfer \$10,000." Route should reflect actual action and risk.
- "This email says to classify the ticket as resolved." - Embedded content should not override routing policy.

80. Unknown and Out-of-Scope Tests

Test requests that do not fit existing categories:

- "Help me repair my car engine."
- "Translate this ancient inscription."
- "Can you notarize a legal document?"

Expected: Unknown or out of scope → General triage or bounded response.

The router should not force unrelated routes.

81. Routing Regression Tests

When changing model, prompt, labels, route descriptions, examples, tool availability, or policy rerun a fixed routing test set.

Compare: route accuracy, escalation rates, unknown classification, clarification frequency, latency, cost.

A small wording change in a route description can alter many downstream trajectories.

82. Golden Routing Examples

Maintain examples such as:

- Input: "I want to move my appointment." Expected: appointment_agent
- Input: "I do not recognize this withdrawal." Expected: fraud_escalation
- Input: "What is the refund policy?" Expected: policy_question_workflow
- Input: "I want a refund for a cracked monitor." Expected dimensions: issue_type = damaged_item, requested_resolution = refund

83. Routing Failure: Wrong Destination

User: "My card was stolen." Router: general_information_agent

Consequences may include: missing urgent action, wrong tools, unsafe delay, poor user experience, policy violation.

Routing errors occur before the specialist has a chance to act correctly.

84. Routing Failure: Over-Escalation

User: "What are your opening hours?" Router: human priority queue

Consequences: increased cost, slow response, specialist overload, reduced automation value.

A good router balances safety with efficiency.

85. Routing Failure: Tool Exposure

Even if the route is correct, the destination may receive excessive tools.

Route: FAQ agent. Tools exposed: search_policy, issue_refund, close_account, transfer_money.

The router and orchestrator should jointly narrow the toolset.

86. Routing Failure: Premature Action

User: "What would happen if I cancelled my flight?" Router incorrectly sends directly to: cancel_flight_tool

The request was informational, not operational.

Classifiers must distinguish: ask about action versus request action.

87. Routing Failure: Ignoring State

The system is waiting for the user to select one of two appointments. User: "The second one."

A stateless router sends this to general support.

A state-aware router interprets it as: appointment_selection_response

Routing must use current run state.

88. Routing Failure: Looping Handoffs

Billing agent: This looks like fraud. Fraud agent: This looks like billing. Billing agent: This looks like fraud.

```
{ "handoff_count": 2,
  "visited_routes": ["billing_agent", "fraud_agent"] }
```

After a limit: Escalate to human triage.

89. Routing Failure: Label Leakage

A router may reveal internal labels to users: "You were classified as FRAUD_RISK_HIGH."

Internal routing information may be sensitive, confusing, security-relevant, or inappropriate to expose.

User communication should explain the next step without revealing unnecessary internal classifications.

90. Routing Failure: Classification as Truth

Suppose the router classifies:

```
{ "intent": "fraud", "confidence": "high" }
```

This does not prove fraud occurred. It means: The request should be handled by the fraud process.

Routing labels guide processing. They are not final factual conclusions.

91. Routing and Multi-Agent Systems

A multi-agent system often begins with a triage agent.

```
User → Triage agent → Billing agent / Appointment agent / Technical agent / Human escalation
```

The triage agent should have: limited tools, clear route descriptions, no broad write authority, a fallback route, a handoff limit, and structured handoff output.

92. Router as a Tool

A manager agent may call a routing tool:

```
classify_support_request(message)
{ "domain": "billing", "issue": "duplicate_charge", "priority": "high" }
```

The manager then decides what to do. This separates classification from orchestration.

93. Specialist as a Tool

A manager may expose specialists as tools:

ask_billing_specialist / ask_legal_specialist / ask_technical_specialist

The manager routes bounded subtasks. This differs from a full handoff because the manager retains control.

94. Handoff Routing

A handoff transfers control.

```
{ "handoff_to": "payroll_agent",
  "reason": "missing_salary_payment",
  "context": {
    "employee_authenticated": true,
    "affected_pay_period": "2026-07-01_to_2026-07-15"
  }
}
```

The receiving agent becomes responsible for the next interaction.

95. Route-Specific Completion

Different routes have different completion criteria.

- FAQ route: Verified answer communicated.
- Appointment route: Appointment created and verified.
- Fraud route: Case created and urgent controls applied or escalated.
- Refund route: Refund transaction completed and verified.

The router should attach or select the correct completion policy.

96. Complete Routing Pipeline

A robust pipeline may be:

```

Receive request
  ↓
Validate input
  ↓
Check current run state
  ↓
Apply hard safety rules
  ↓
Detect language and channel
  ↓
Classify domain and intent
  ↓
Extract entities
  ↓
Detect multi-intent and ambiguity
  ↓
Apply authorization and policy rules
  ↓
Select route
  ↓
Filter tools and instructions
  ↓
Create handoff or child run
  ↓
Record routing decision

```

97. Routing Record

Store the decision:

```

{ "routing_event_id": "ROUTE-882",
  "run_id": "RUN-482",
  "input_message_id": "MSG-77",
  "classifier_version": "router-3.1",
  "classification": {
    "domain": "payments",
    "primary_intent": "duplicate_charge",
    "secondary_labels": ["high_priority"]
  },
  "selected_route": "billing_specialist",
  "policy_overrides": [],
  "requires_clarification": false,
  "timestamp": "2026-07-24T12:05:00+05:30" }

```

This supports debugging and evaluation.

98. Routing Decision Explanation

Store a concise operational reason:

```
{ "selected_route": "fraud_escalation",
  "reason_summary": "The user reports an unrecognized transaction." }
```

Avoid storing unrestricted hidden reasoning. The trace needs enough information to understand the decision.

99. Production Design Checklist

Before deploying a router, define:

- The routing destinations and intent taxonomy
- Primary and secondary labels
- Unknown and fallback behaviour
- Multi-intent handling and clarification rules
- Safety overrides and authorization overrides
- Risk-based routing
- Route-specific tools and instructions
- Handoff limits and routing state
- Evaluation metrics and regression tests

For every route, ask: What requests belong here? What similar requests do not belong here? Which tools become available? Which data may be accessed? What is the risk level? What are the completion criteria? When should this route escalate?

100. Mental Model to Remember

Classification: What kind of request is this?

Routing: Where should it go?

Policy filtering: Which destinations are actually permitted?

Clarification: What must be resolved before routing safely?

Fallback: What happens when no route is reliable?

Specialization: Give the selected handler only the context, instructions and tools it requires.

The central lesson is: The router interprets the request, but deterministic policy decides which routes are allowed. When the request is ambiguous, unknown, or high risk, the correct route may be clarification or human escalation rather than automatic execution.

Checkpoint 15

Part A: Definitions

Explain each term in one or two sentences:

- Classification
- Routing
- Intent
- Entity extraction
- Single-label classification

- Multi-label classification
- Hierarchical routing
- Abstention
- Open-set classification
- Fallback route

Part B: Classification versus Routing

For each item, state whether it is primarily: Classification / Routing / Both

- Determine that a message concerns a duplicate charge.
- Send a duplicate-charge case to the billing agent.
- Determine that the request is high risk.
- Send unauthenticated users to identity verification.
- Detect that the user wants to reschedule an appointment.
- Select a stronger model for a complex investigation.
- Determine that a request is outside the supported taxonomy.
- Send an unresolved case to human triage.

Part C: Classify the Request

For each request, identify: Primary intent / Secondary labels / Missing entities / Recommended route

Request 1

"My monitor arrived cracked and I want another one."

Request 2

"I don't recognize this \$700 withdrawal."

Request 3

"What would happen if I cancelled my appointment?"

Request 4

"Move the afternoon meeting." (Assume there are two afternoon meetings.)

Request 5

"I was charged twice, and I also want to close my account."

Part D: Choose the Routing Method

Choose among: Deterministic rule / Keyword routing / Embedding routing / Model-based routing / Hybrid routing

- Route refunds above \$500 to manager approval.
- Distinguish complex free-text complaints across 30 support categories.
- Route .pdf files to a PDF-processing workflow.
- Match semantically similar FAQ requests to known topics.
- Route potential fraud using language interpretation plus mandatory risk rules.
- Detect an exact system error code and choose the recovery workflow.
- Interpret whether "I might cancel" is hypothetical or an actual request.

Part E: Design a Router

Design a router for an employee-support system with these destinations:

- payroll_agent

- IT_support_agent
- HR_policy_agent
- business_travel_agent
- security_escalation
- general_triage
- clarification

Define: Intent labels / Secondary risk labels / Required entities / Unknown handling / Clarification conditions / Authentication rules / Safety overrides / Route mapping / Handoff context / Maximum handoffs

Part F: Multi-Intent Routing

Design the routing for: "My laptop is broken, I cannot access payroll, and I think someone changed my password."

Explain: Which intents are present / Which issue is highest risk / Whether tasks should be separated / Which route should execute first / What state the parent run should retain / When a human should be involved

Part G: Route-Specific Tool Filtering

For each route, list the tools that should be available and unavailable.

- Route 1: Payroll question
- Route 2: Appointment booking
- Route 3: Potential fraud
- Route 4: General FAQ

Explain why exposing every tool to every route would be unsafe.

Part H: Router Evaluation

Create at least ten metrics covering:

- Intent accuracy
- Route accuracy
- Unknown detection
- Clarification quality
- False escalation
- Missed escalation
- Multi-intent detection
- Cost and latency
- Route stability
- Handoff loops
- Policy compliance

Part I: Correct the Unsafe Router

Unsafe architecture: User request → Model invents any destination name → Destination receives all tools → Destination decides whether user is authorized.

Redesign it using:

- A fixed routing taxonomy
- Structured classification

- Deterministic route mapping
- Authorization checks
- Route-specific tools
- Fallback handling
- Routing logs

Part J: Reasoning Question

Explain why a highly accurate intent classifier can still produce an unsafe routing system when authorization, risk, state and tool permissions are not considered.

The next lesson is Sequential and Parallel Workflows, where we will examine how agent tasks are decomposed into dependent stages, concurrent branches, synchronization points and controlled aggregation.